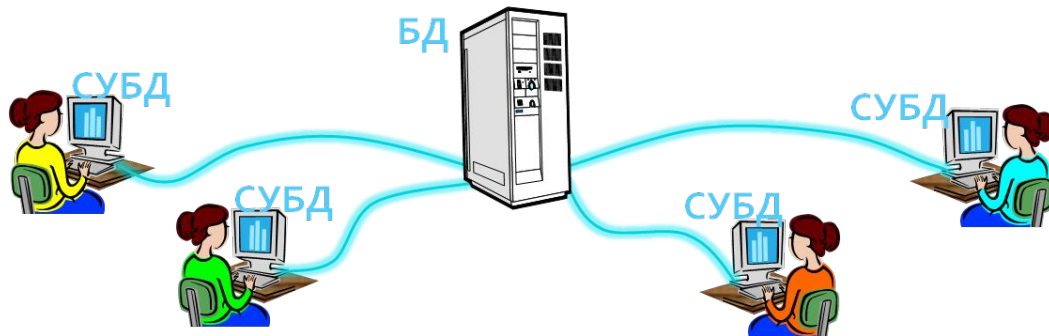

Введение в Базы Данных

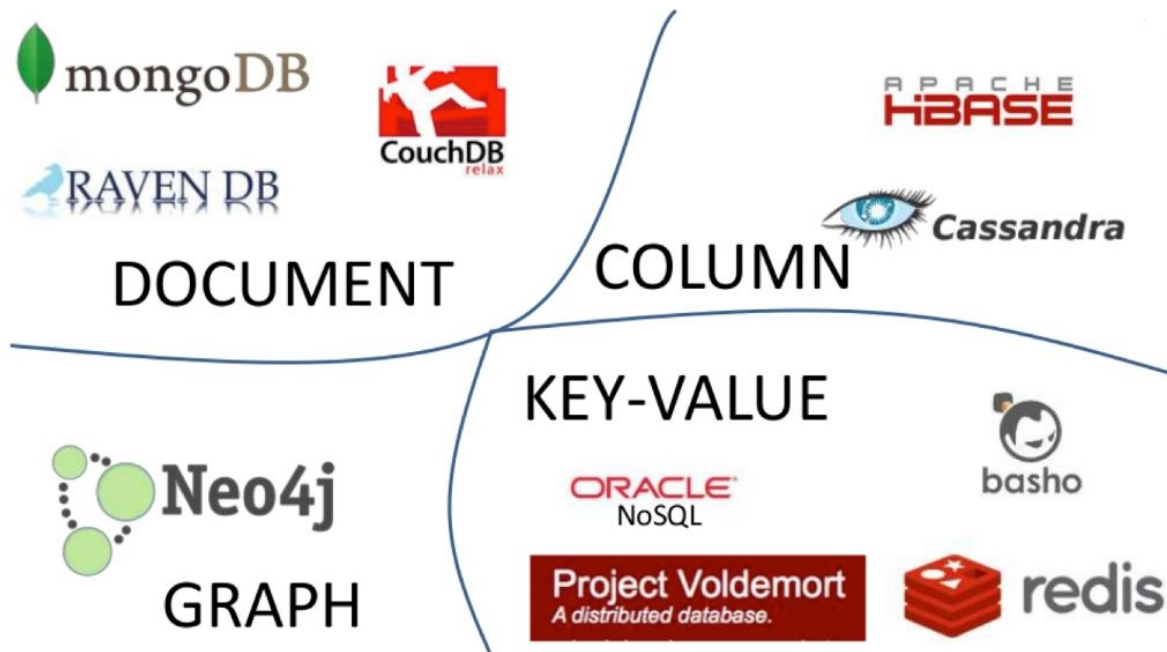
— Теория —

Основные понятия

- ▶ База данных (БД) — именованная совокупность данных, отражающая состояние объектов и их отношений в рассматриваемой предметной области.
- ▶ Система управления базами данных (СУБД) — совокупность языковых и программных средств, предназначенных для создания, наполнения, обновления и удаления баз данных.
- ▶ Модель данных — это некоторая абстракция, которая, будучи приложима к конкретным данным, позволяет пользователям и разработчикам трактовать их уже как информацию, то есть сведения, содержащие не только данные, но и взаимосвязь между ними.



NoSQL (нереляционные базы данных)



<https://www.youtube.com/watch?v=IBzTDkYNB7I>

Реляционная модель данных

Понятие «реляционный» основано на англ. relation («отношение, зависимость, связь»)

Normalization (нормализация) это процесс организации данных, который включает создание таблиц и установление связей между ними в соответствии с правилами, которые обеспечивают защиту данных и делают базу данных более гибкой, устраняя избыточность и несогласованные зависимости.

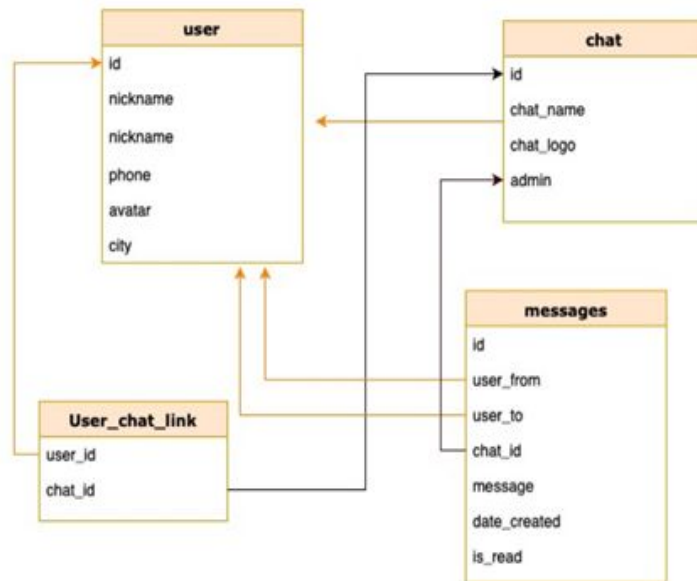
User_from	User_from_p hone	User_to_nic k	User_to_ph one	Chat	Message	datetime	
Vasya	9999	Petya	0000	xxx	Hey!	2020-01-01 00:00:00	
Petya	0000	Vasya	9999	xxx	Hey!	2020-01-01 00:00:01	
Vasya	9999	Petya	0000	xxx	How R U	2020-01-01 00:00:02	
Petya	0000	Vasya	9999	xx	fine	2020-01-01 00:00:05	

Нормализация

User_from	User_from_phone	User_to_nickname	User_to_phone	Chat	Message	datetime	
Vasya	9999	Petya	0000	xxx	Hey!	2020-01-01 00:00:00	
Petya	0000	Vasya	9999	xxx	Hey!	2020-01-01 00:00:01	
Vasya	9999	Petya	0000	xxx	How R U	2020-01-01 00:00:02	
Petya	0000	Vasya	9999	xx	fine	2020-01-01 00:00:05	

Нормализация

- + Упрощение системы
- + Уменьшение количества ошибок
- + Устранение избыточности данных
- + Уменьшение размера БД
- + Устранение несогласованных зависимостей



Нормализация

Существует 6 основных нормальных форм базы данных:

- **Первая нормальная форма (1NF)**
- **Вторая нормальная форма (2NF)**
- **Третья нормальная форма (3NF)**
- Нормальная форма Бойса-Кодда (BCNF)
- Четвертая нормальная форма (4NF)
- Пятая нормальная форма (5NF)
- Доменно-ключевая нормальная форма (DKNF)
- Шестая нормальная форма (6NF)

1 НФ

Определение:

Переменная отношения находится в первой нормальной форме (1НФ) тогда и только тогда, когда в любом допустимом значении отношения каждый его кортеж содержит только одно значение для каждого из атрибутов.

Пояснение:

В каждой клеточке таблицы должно быть только одно значение. Не должно быть повторяющихся строк.

1 НФ - до нормализации

name	hobbies
Суворов Алексей	Коньки, Мотоцикл
Окулов Ярослав	ClickHouse
Созинов Илья	Портреты, PlayStation
Белянин Алексей	Фотография, Велосипеды, Аудиотехника, Ножи
Збруев Роман	Гитара, Радиотехника

1 НФ - после нормализации

name	hobbie
Суворов Алексей	Коньки
Суворов Алексей	Мотоцикл
Окулов Ярослав	ClickHouse
Созинов Илья	Портреты
...	...
Збруев Роман	Радиотехника

2 НФ

Определение:

Переменная отношения находится во второй нормальной форме тогда и только тогда, когда она находится в первой нормальной форме и каждый неключевой атрибут неприводимо (функционально полно) зависит от её потенциального ключа.

Пояснение:

Таблица в 1НФ

Есть первичный ключ

Все атрибуты зависят от первичного ключа целиком, а не от какой-то его части

2 НФ - до нормализации

name	project	project_client
Суворов Алексей	КайзерДом	Сергей И.
Суворов Алексей	Юнитраст	Иван С.
Окулов Ярослав	Доска почета	Аркадий Г.
Созинов Илья	КайзерДом	Сергей И.
Збруев Роман	КайзерДом	Сергей И.
Збруев Роман	Юнитраст	Иван С.

2 НФ - добавим первичный ключ

worker_id	project_id	name	project	project_client
1	1	Суворов Алексей	КайзерДом	Сергей И.
1	2	Суворов Алексей	Юнитраст	Иван С.
2	3	Окулов Ярослав	Доска почета	Аркадий Г.
3	1	Созинов Илья	КайзерДом	Сергей И.
4	1	Збруев Роман	КайзерДом	Сергей И.
4	2	Збруев Роман	Юнитраст	Иван С.

2 НФ - Декомпозиция

project_id	project	project_client
1	КайзерДом	Сергей И.
2	Юнитраст	Иван С.
3	Доска почета	Аркадий Г.

worker_id	project_id
1	1
1	2
2	3
3	1
4	1
4	2

worker_id	name
1	Суворов Алексей
2	Окулов Ярослав
3	Созинов Илья
4	Збруев Роман

3 НФ

Определение:

Переменная отношения находится в третьей нормальной форме тогда и только тогда, когда она находится во второй нормальной форме, и отсутствуют транзитивные функциональные зависимости неключевых атрибутов от ключевых.

Пояснение:

Таблица в 2 НФ

Все атрибуты зависят только от первичного ключа, но не от других атрибутов

3 НФ - До нормализации

tyre_id	tyre_name	supplier	supplier_phone
1	Nokian Hakka	ИП Шлепаков Л.	+79233213123
2	Nokian Nordman	ООО Русский Север	+79242343423
3	Continental Contilce	ООО Автомакс	+79434234234
4	Matador	ИП Кузнецов П.	+79989493248
5	Nordmaster	ИП Иванов П.	+79646564345

3 НФ - После нормализации

tyre_id	tyre_name	supplier_id
1	Nokian Hakka	1
2	Nokian Nordman	2
3	Continental Contilce	3
4	Matador	4
5	Nordmaster	5

id	supplier	supplier_phone
1	ИП Шлепаков Л.	+79233213123
2	ООО Русский Север	+79242343423
3	ООО Автомакс	+79434234234
4	ИП Кузнецов П.	+79989493248
5	ИП Иванов П.	+79646564345

Индексы

Индексы

chat_id
11
23
45
67

Messages

id	chat_id	user_from	user_to	message	dt_created	is_read
0	11	1	2	'msg'	2020-01-01 00:00:00	1
1	11	2	1	'msg'	2020-01-01 00:00:00	0
2	23	3	4	'msg'	2020-01-01 00:00:00	1
3	45	4	3	'msg'	2020-01-01 00:00:00	1
4	67	5	6	'msg'	2020-01-01 00:00:00	0

Типы индексов

- Кластерный индекс . В таблице только один. Строки таблицы упорядочены по значению этого ключа.
- Некластерный . Содержит только указатель на строки таблицы. Важен порядок ключевых полей индекса.

Таблица без индексов называется кучей

Индексирование

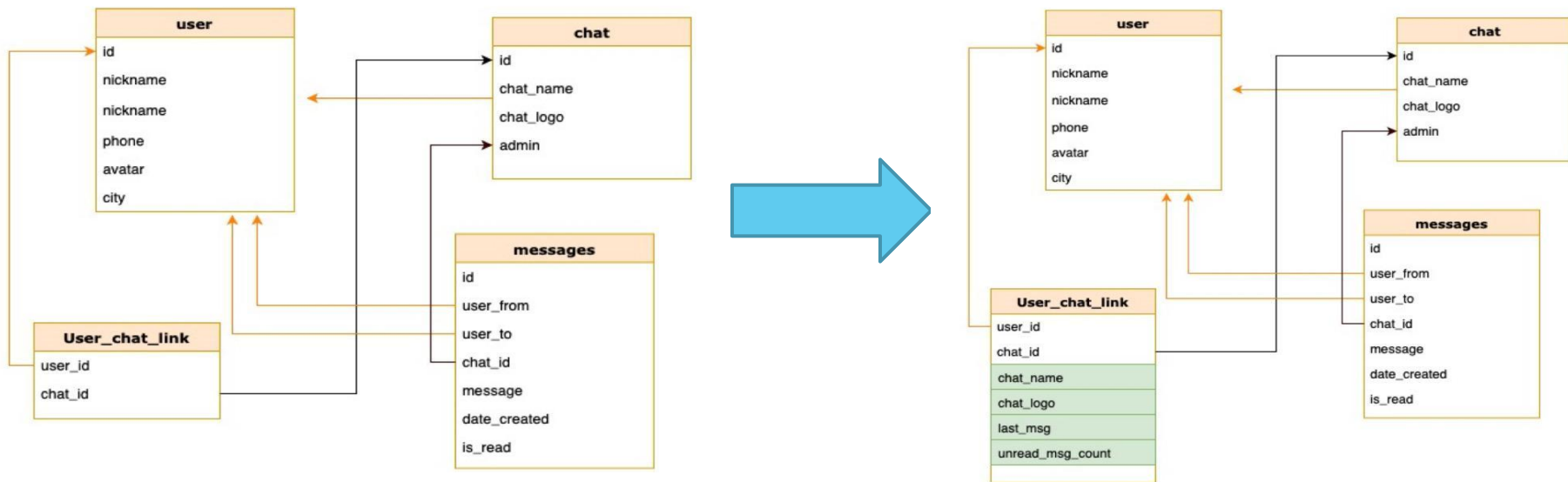
Внимание на

! Накладные расходы на поддержание

! Высокая селективность индекса

! Возможна фрагментация

Денормализация



Денормализация

- + Ускорение процессов выборки данных

- + Упрощение написания запросов выборки

- ! Дорогие запросы обновления данных (create/update/delete)

- ! Сложность написания кода обновления данных, повышение вероятности ошибок

- ! Сначала нормализовать

- ! Сначала проверить оптимальность запросов

Транзакции

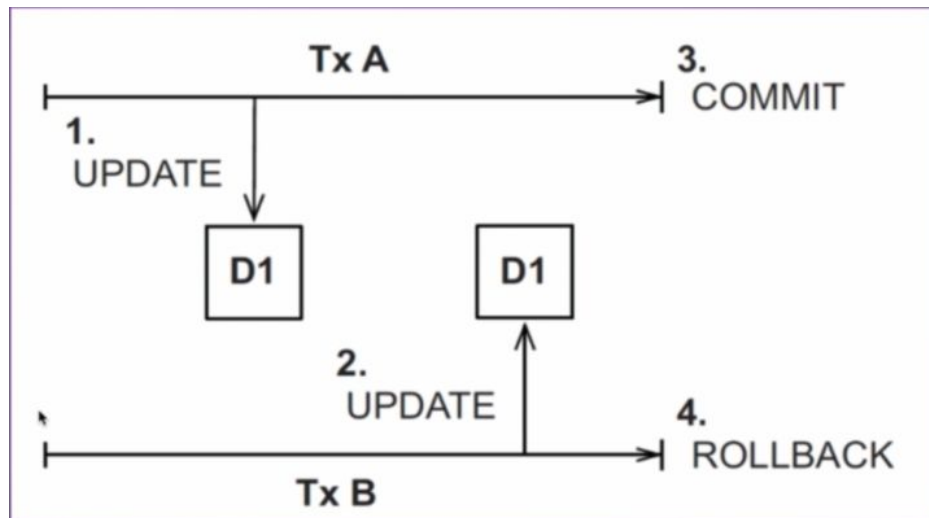


ACID

- ▶ Atomicity (Атомарность)
 - ▶ атомарность гарантирует, что ни одна транзакция не будет частично зафиксирована в системе
- ▶ Consistency (Согласованность)
 - ▶ каждая успешная транзакция по определению фиксирует только допустимые результаты, сохраняет согласованность базы данных
- ▶ Isolation (Изолированность)
 - ▶ во время транзакции параллельные транзакции не должны влиять на ее результат
- ▶ Durability (Устойчивость)
 - ▶ независимо от проблем на более низких уровнях (например, обесточивание системы или неисправности оборудования), изменения, внесенные в результате успешно выполненной транзакции, должны оставаться в памяти после того, как система вернется к работе

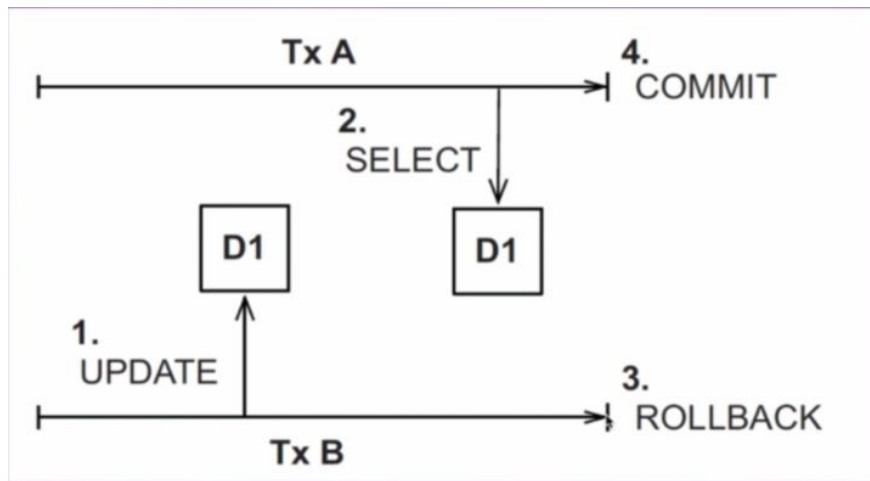
Lost Update Problem

Происходит , когда обе транзакции одновременно обновляют данные и затем вторая транзакция откатывает изменения , вследствие чего изменения обеих транзакций теряются



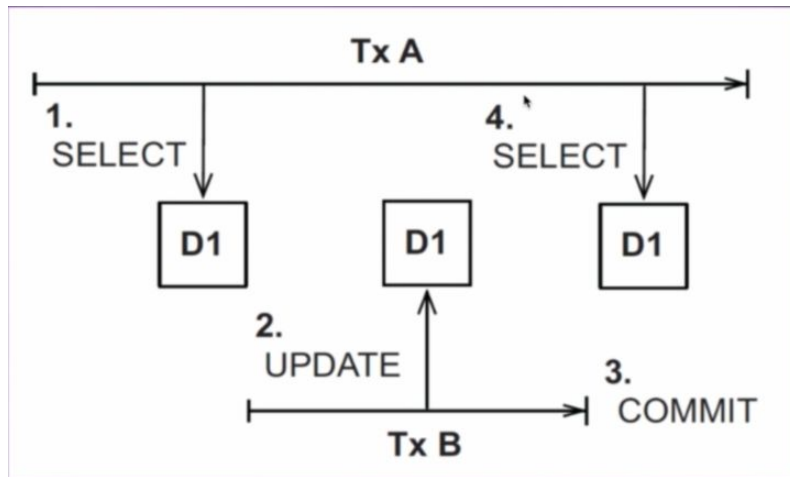
Dirty Read Problem

Происходит, когда первая транзакция читает изменения , сделанные другой транзакцией , но эти изменения еще не были ей закомичены. После чего вторая транзакция откатывает эти изменения , а первая продолжает работу с “грязными” данными.



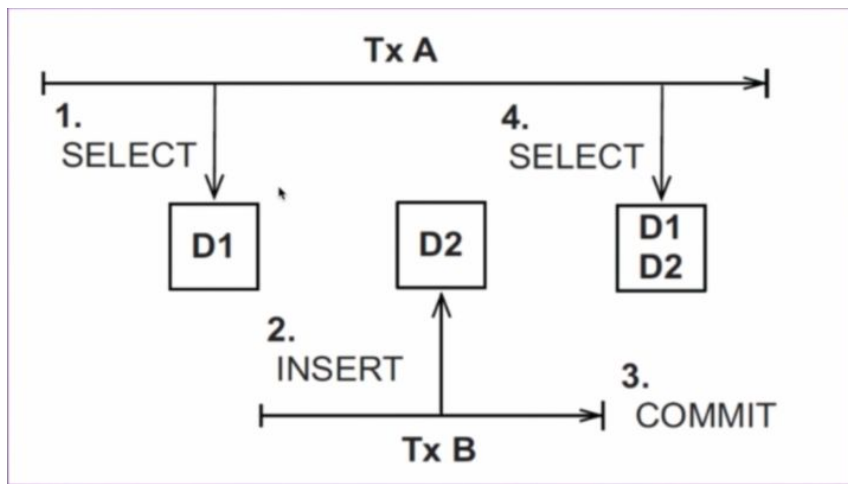
Non Repeatable Read Problem

Происходит, когда первая транзакция читает одни и те же данные дважды, но после первого прочтения вторая транзакция изменяет (update) эти же данные и делает коммит, вследствие чего вторая выборка в первой транзакции вернет другой результат.



Phantom read

Происходит, когда первая транзакция читает одни и те же данные дважды, но после первого прочтения вторая транзакция добавляет новые строки или удаляет старые и делает коммит, вследствие чего вторая выборка в первой транзакции вернет другой результат(разное количество записей)



Read uncommitted (чтение незафиксированных данных)

NOLOCK

Блокировка записи данных на время выполнения изменения, т.е. команды изменения одних и тех же строк выполняются последовательно

**Транзакции на чтение не блокируются.
Возможно считывание не только логически несогласованных данных, но и данных, изменения которых ещё не зафиксированы.**



Пример

дебаг

Read committed (чтение фиксированных данных)

Защищает от:



«грязное» чтение

Блокирует строки на чтение, пока с ними работает транзакция

Гарантирует, что незафиксированные данные не будут прочитаны

Не блокирует таблицу от изменений другими транзакциями.



Пример

По умолчанию. Недолгие транзакции, без промежуточных этапов и аналитики

Repeatable read (повторяемость чтения)

Защищает от:

- ✓ потерянное обновление (*lost update*)
- ✓ «грязное» чтение
- ✓ неповторяющееся чтение (*non-repeatable read*)

Гарантирует, что незафиксированные данные не будут прочитаны, а данные, находящиеся в обработке, не будут изменены другими транзакциями.

Степень параллелизма ниже, чем при уровне изоляции READ COMMITTED.



Пример

каждая транзакция видит полностью стабильное представление базы данных

Serializable (упорядочиваемость)

Защищает от:

- ✓ потерянное обновление (*lost update*)
- ✓ «грязное» чтение
- ✓ неповторяющееся чтение (*non-repeatable read*)
- ✓ фантомное чтение (*phantom reads*)

Все транзакции обрабатываются так, как будто они все запущены последовательно (одна за другой).

Отслеживание зависимостей чтения-записи неизбежно создаёт дополнительную нагрузку, как и перезапуск транзакций

✓ Пример

самый высокий уровень изолированности

Уровни изоляции транзакций

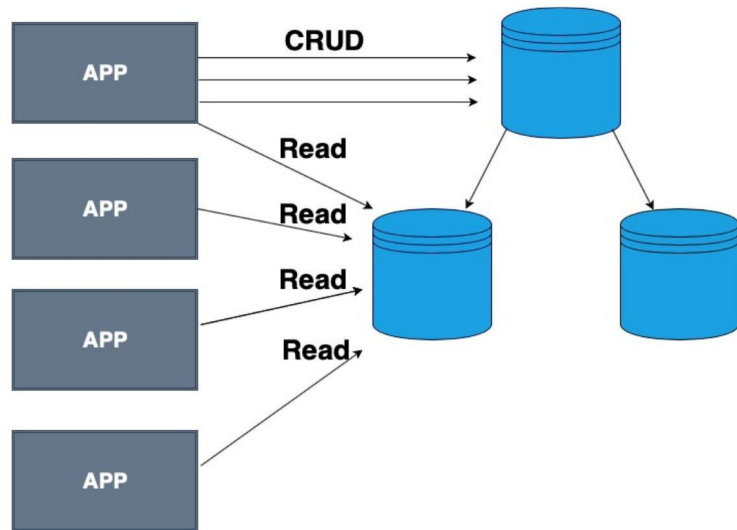
		Isolation Level			
		Read Uncommitted	Read Committed	Repeatable Read	Serializable
Problem Type	Dirty Read	Possible	Not Possible	Not Possible	Not Possible
	Nonrepeatable Read	Possible	Possible	Not Possible	Not Possible
	Phantom Read	Possible	Possible	Possible	Not Possible

Репликация

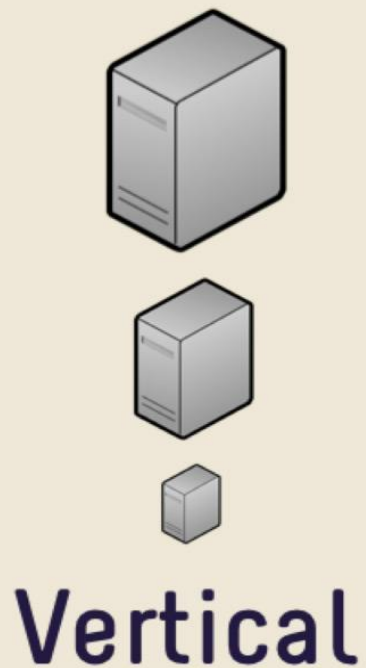
Репликация — одна из техник масштабирования баз данных. Состоит эта техника в том, что данные с одного сервера базы данных постоянно копируются (реплицируются) на один или несколько других (называемые репликами).

Задачи, решаемые репликацией:

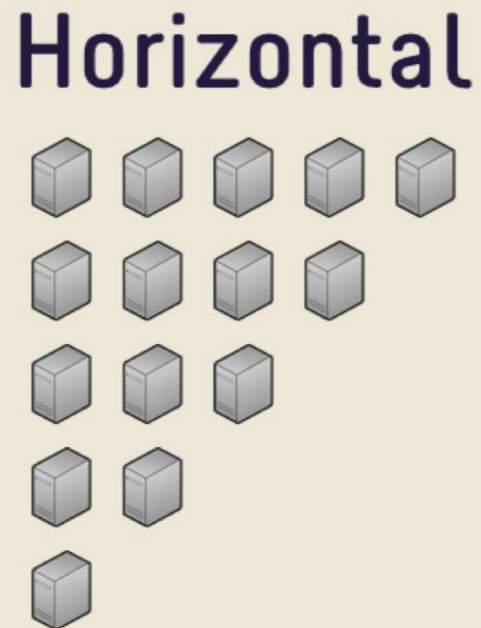
- поддержка резервной базы данных на случай потери основной;
- снижение нагрузки на базу за счёт переноса части запросов на реплики;
- перенос данных в архивные или аналитические системы.



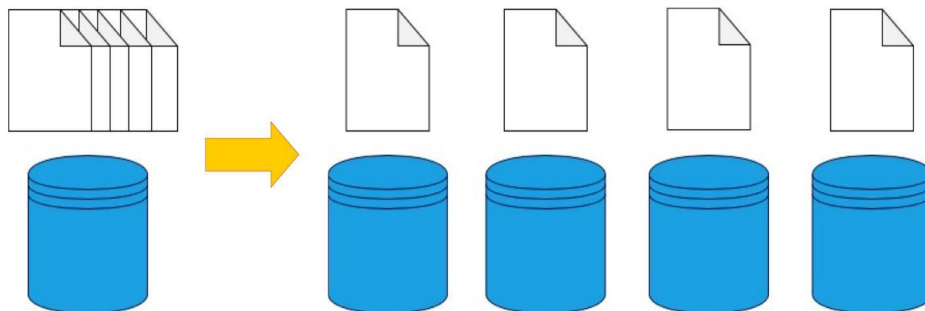
Масштабирование



vs.



Шардинг



PRODUCT	PRICE
WIDGET	\$118
GIZMO	\$88
TRINKET	\$37
THINGAMAJIG	\$18
DOODAD	\$60
TCHOTCHKE	\$999



(\$0-\$49.99)

PRODUCT	PRICE
TRINKET	\$37
THINGAMAJIG	\$18

(\$50-\$99.99)

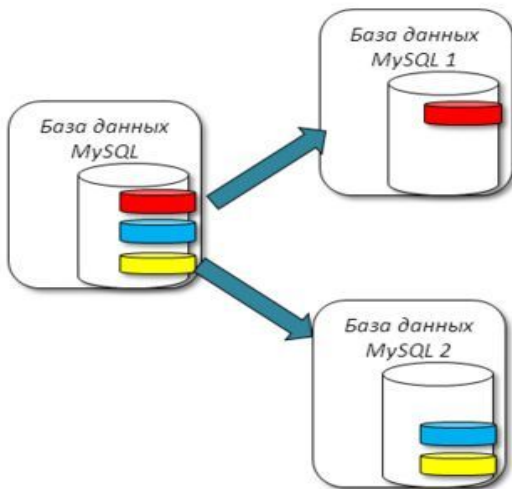
PRODUCT	PRICE
GIZMO	\$88
DOODAD	\$60

(\$100+)

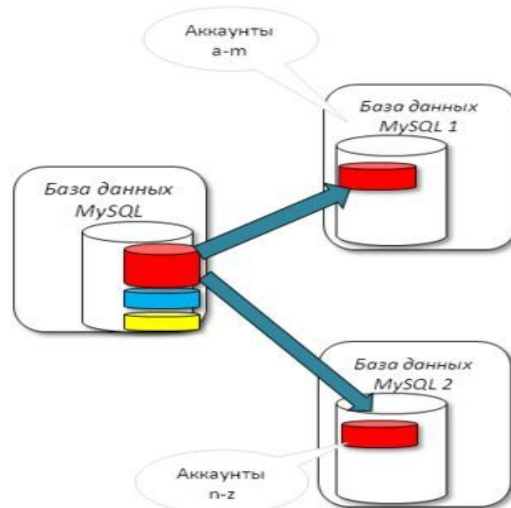
PRODUCT	PRICE
WIDGET	\$118
TCHOTCHKE	\$999

Шардинг

Вертикальный шардинг



Горизонтальный



<https://www.youtube.com/watch?v=ECkQtFnOxuA>

Шардинг

- + Масштабирование
- + Большой размер данных
- ! Равномерная загрузка узлов
- ! Равномерное распределение данных
- ! Ключ секционирования

Команды SQL

DDL

язык определения данных

ALTER
COLLATE
CREATE
DROP
DISABLE TRIGGER
ENABLE TRIGGER
RENAME
UPDATE STATISTICS

DML

язык манипулирования данными

BULK INSERT
SELECT
DELETE
UPDATE
INSERT
UPDATETEXT
MERGE
WRITETEXT
READTEXT

DCL

язык управления данными

GRANT
REVOKE
DENY

TCL

язык управления транзакциями

BEGIN
COMMIT
ROLLBACK

PRIMARY key

- ▶ Ограничение PRIMARY KEY однозначно идентифицирует каждую запись в таблице базы данных
- ▶ Первичные ключи должны содержать значения UNIQUE и не могут содержать значения NULL
- ▶ Таблица может иметь только один первичный ключ, который может состоять из одного или нескольких полей

On CREATE table:

```
CREATE TABLE Author(
```

```
    id_author int NOT NULL PRIMARY KEY,
```

```
.....
```

```
);
```

FOREIGN key

- ▶ FOREIGN KEY - это ключ, используемый для связывания двух таблиц
- ▶ FOREIGN KEY - это поле (или набор полей) в одной таблице, которое ссылается на PRIMARY KEY в другой таблице
- ▶ Таблица, содержащая внешний ключ, называется дочерней таблицей, а таблица, содержащая ключ-кандидат, называется ссылочной или родительской таблицей

On CREATE table:

```
CREATE TABLE book(
```

```
id_book int NOT NULL PRIMARY KEY,
```

```
FOREIGN KEY (id_book_type) REFERENCES book_type(id_book_type)
```

```
);
```


DROP table

Оператор DROP TABLE используется для удаления существующей таблицы в базе данных SQL

Синтаксис:

```
DROP TABLE tablename;
```

Пример:

```
DROP TABLE demo;
```

CREATE table

Оператор **CREATE TABLE** используется для создания новой таблицы в базе данных.

Параметры столбца указывают имена столбцов таблицы.

Параметр типа данных указывает тип данных, которые может содержать столбец (например, varchar, integer, date и т. д.).

Синтаксис:

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    ....  
);
```

Пример:

```
CREATE TABLE book (  
    id_book serial not null constraint book_pk primary key,  
    author varchar(255),  
    ...)
```

INSERT INTO table

Инструкция INSERT INTO используется для вставки новых записей в таблицу

Синтаксис:

```
INSERT INTO table_name (column1, column2, column3, ...)
```

```
VALUES (value1, value2, value3, ...);
```

Или

```
INSERT INTO table_name VALUES (value1, value2, value3, ...);
```

Пример:

```
INSERT INTO public.author (full_name, birth_year, death_year)
```

```
VALUES ('Джек Лондон', '1876-01-12', '1916-11-22');
```

UPDATE table

Оператор **UPDATE** используется для изменения существующих записей в таблице

Синтаксис:

```
UPDATE table_name SET column1 = value1, column2 = value2, ...  
WHERE condition;
```

Пример:

```
UPDATE public.author SET public.author.full_name= 'DJEk LONDON' WHERE  
public.author.id_author= 4;
```

DELETE from table

Оператор **DELETE** используется для удаления существующих записей в таблице

Синтаксис:

```
DELETE FROM table_name WHERE condition;
```

Пример:

```
DELETE FROM public.author WHERE full_name='Джек Лондон';
```

SELECT from one table

Оператор SELECT используется для выбора данных из базы данных.

Возвращенные данные хранятся в таблице результатов, которая называется набором результатов

Синтаксис:

```
SELECT column1, column2, ... FROM table_name;
```

Пример:

```
SELECT * FROM public.author;
```

SELECT from few table

Синтаксис:

```
SELECT column1, column2, ... FROM table_name1, table_name2;  
WHERE table_name1.FOREIGN_KEY = table_name2.PRIMARY_KEY
```

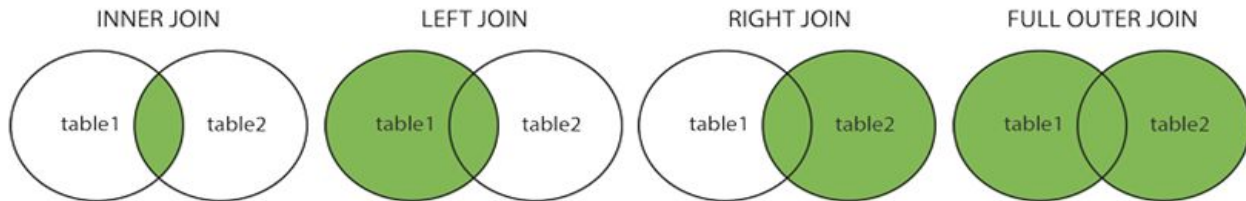
Пример:

```
SELECT public.author.full_name, public.book.title FROM public.author, public.book WHERE  
public.author.id_book=public.book.id_book;
```

Joins

Предложение JOIN используется для объединения строк из двух или более таблиц на основе связанного столбца между ними

- ▶ **(INNER) JOIN**: Возвращает записи, которые имеют совпадающие значения в обеих таблицах
- ▶ **LEFT (OUTER) JOIN**: Вернуть все записи из левой таблицы и сопоставленные записи из правой таблицы
- ▶ **RIGHT (OUTER) JOIN**: Возврат всех записей из правой таблицы и сопоставленных записей из левой таблицы
- ▶ **FULL (OUTER) JOIN**: Вернуть все записи, когда есть совпадение в левой или правой table



Общая структура запроса SELECT

SELECT ('столбцы или * для выбора всех столбцов; обязательно')

FROM ('таблица; обязательно')

WHERE ('условие/фильтрация, например, city = 'Moscow'; необязательно')

GROUP BY ('столбец, по которому хотим сгруппировать данные; необязательно')

HAVING ('условие/фильтрация на уровне сгруппированных данных; необязательно')

ORDER BY ('столбец, по которому хотим отсортировать вывод; необязательно')

Домашнее задание:

1. Развернуть базу данных на своём компьютере одним из двух способов:

А. Установить БД можно и напрямую в Windows через скачивание самой БД: [PostgreSQL: Windows installers](#) (нужен VPN). Доступ в PGAdmin - запускается, как обычное приложение для Windows.

Б. По примеру статьи ([БД в Docker](#)) развернуть **Базу данных** и **PGAdmin***. Для этого необходимо установить Docker ([Docker Desktop](#)). Доступ в PGAdmin после установки и запуска можно получить по ссылке <http://localhost:5050/browser/>

2. Создать/удалить несколько таблиц (с наличием primary key, с наличием foreignkey).

3. Выполнить команды INSERT, UPDATE, DELETE на созданных таблицах для манипуляции со строками.

4. Научиться выполнять запросы SELECT, как простые, так и с JOIN.

5. Удалить созданные таблицы командой DROP.

6. Пройти 1 главу курса [Интерактивный тренажер по SQL](#)

* - Для доступа к БД можно также использовать **DBeaver**, **DataGrip**.

Источники

[Про SQL за 7 минут \(видео\)](#)

[БД в Docker](#)

[Интерактивный тренажер по SQL](#)

[SQLite Online](#)

[Плейлист по БД \(R class Tech\)](#)

[PostgreSQL | Введение](#)